PEMROGRAMAN BERORIENTASI OBJEK 2026

# *Design Pattern* Dasar

Resep teruji untuk masalah desain yang terus muncul: *Singleton*, *Factory*, *Strategy*, dan *Observer*

PERTEMUAN

14

dari 16

Program Studi Informatika · Universitas Mulawarman
Jalur praktik: Python dan Java

# Tujuan Pembelajaran

Setelah pertemuan ini, kamu mampu:

1. Menjelaskan apa itu ***design pattern***, asal-usulnya (buku GoF), dan tiga kategorinya.
2. Menjelaskan masalah, tujuan, dan struktur kelas dari pola ***Singleton***, ***Factory***, ***Strategy***, dan ***Observer***.
3. Membedakan *Simple Factory* dengan *Factory Method*, serta gaya *push* dengan *pull* pada *Observer*.
4. Menerapkan keempat pola tersebut pada program kecil dan pada studi kasus KedaiKita.
5. Menilai kapan sebuah pola tepat dipakai dan kapan justru membuat program rumit, termasuk kekurangan *Singleton*.

# Mengingat Kembali



**Dari pertemuan sebelumnya**

- **SRP**: `StrukPesanan` hanya mengurus format struk; aturan hitung harga tetap di `Pesanan`.
- **OCP**: setiap aturan diskon menjadi kelas turunan `Diskon`; promo baru cukup ditambah sebagai kelas baru.
- **DIP**: `Kedai` menerima `PenyimpananPesanan` lewat konstruktor (*dependency injection*).
- SOLID menjawab “kode yang baik itu seperti apa?”; *design pattern* menjawab “untuk masalah ini, susunan kelas apa yang terbukti berhasil?”

# Agenda Hari Ini

1. Apa itu *design pattern*?
2. *Singleton*: hanya boleh ada satu
3. *Factory*: serahkan urusan membuat objek
4. *Strategy*: algoritma yang bisa ditukar
5. *Observer*: kabari semua yang berlangganan
6. Studi kasus KedaiKita

BAGIAN 1

1

# Apa Itu *Design Pattern*?

Resep yang sudah teruji untuk masalah desain yang terus muncul.

# Pola Jahit di Penjahit Langganan

**Analogi**

- Di dinding kios penjahit tergantung **pola jahit**: kertas cokelat bergambar potongan lengan, kerah, dan badan kemeja.
- Pola kemeja **bukan kemeja**. Pola yang sama dipakai untuk banyak pelanggan, tetapi **ukurannya disesuaikan**.
- Pola lahir dari **pengalaman bertahun-tahun**. Sesama penjahit cukup menyebut namanya dan langsung paham.

| Dunia penjahit | Dunia pemrograman |
|---|---|
| Pola jahit | ***Design pattern*** (pola desain) |
| Pola bukan kemeja | Pola bukan kode jadi |
| Ukuran Budi dan Siti berbeda | Pola disesuaikan dengan programmu |
| “Pakai pola kemeja lengan panjang” | Nama pola menjadi kosakata bersama |

# *Design Pattern*: Solusi yang Sudah Teruji

## *Design pattern* (pola desain)

Solusi umum yang **sudah teruji** untuk masalah desain yang sering muncul berulang kali. Pola **bukan kode jadi**, melainkan gambaran susunan kelas dan objek yang kamu sesuaikan dengan programmu.

- Dipopulerkan buku *Design Patterns* (1994) karya ***Gang of Four*** (GoF): **23 pola** dalam **3 kategori**.
- *Creational* (cara objek dibuat), *structural* (cara kelas dan objek dirangkai), *behavioral* (cara objek berbagi tugas).
- Hari ini: *Singleton* dan *Factory* (*creational*), *Strategy* dan *Observer* (*behavioral*).

BAGIAN 2

2

# *Singleton*: Hanya Boleh Ada Satu

Pola *creational* untuk objek yang memang hanya boleh ada satu di seluruh program.

# Satu Mesin Nomor Antrean

### Analogi

- **Masalah:** pajak diubah menjadi 11%, tetapi hanya sampai ke salah satu objek pengaturan. Struk tertulis `Pajak 10%`, padahal hitungannya memakai 11%.
- Jika Pintu Utara dan Pintu Selatan punya mesin antrean sendiri, bisa ada dua orang memegang nomor **A005**.
- ***Singleton*** menjamin sebuah kelas **hanya punya satu objek** dan menyediakan **satu pintu akses** ke objek itu.

| Kantor pelayanan | Pola *Singleton* |
|---|---|
| Satu mesin untuk semua pintu | Satu objek untuk seluruh program |
| Semua tombol terhubung ke mesin itu | Satu pintu akses: `get_instance()` |
| Nomor antrean tidak pernah kembar | Semua bagian membaca data yang sama |

# `PengaturanKedai`: Pengaturan yang Hanya Ada Satu

Studi kasus KedaiKita

Java menutup konstruktor dengan `private`. Python memasang **penjaga** di `__init__` yang menolak objek kedua.

**Python** python/studi-kasus/pengaturan.py

```python
class PengaturanKedai:
    _instance = None
    def __init__(self):
        if PengaturanKedai._instance is not None:
            # ... raise RuntimeError: objek kedua ditolak
        self._nama_kedai = "KedaiKita"
        self._alamat = "Jl. Pramuka No. 10, Samarinda"
        self._pajak_persen = 10

    @classmethod
    def get_instance(cls) -> "PengaturanKedai":
        if cls._instance is None:
            cls._instance = cls()
        return cls._instance
```

**Java** java/studi-kasus/PengaturanKedai.java

```java
public class PengaturanKedai {
    private static PengaturanKedai instance = null;
    // ... (dilewati)
    private PengaturanKedai() {
        this.namaKedai = "KedaiKita";
        this.alamat = "Jl. Pramuka No. 10, Samarinda";
        this.pajakPersen = 10;
    }
    public static PengaturanKedai getInstance() {
        if (instance == null) {
            instance = new PengaturanKedai();
        }
        return instance;
    }
    // ... (dilewati)
}
```

# Kelebihan dan Kekurangan *Singleton*

| Kelebihan | Kekurangan |
|---|---|
| Satu sumber kebenaran: semua bagian membaca data yang sama | *State* global: perubahan di satu tempat diam-diam memengaruhi tempat lain |
| Objek bisa dibuat hanya saat dibutuhkan (*lazy*) | Ketergantungannya tersembunyi: tidak terlihat dari parameter *method* |
| Mudah diakses tanpa mengoper objek ke mana-mana | Sulit diuji: uji yang satu bisa mengubah hasil uji berikutnya |

Jangan memakai *Singleton* hanya karena malas mengirim parameter. *Dependency injection* lebih mudah diuji.

BAGIAN 3

3

# *Factory*: Serahkan Urusan Membuat Objek

Pola *creational* kedua: pemanggil cukup menyebut *apa* yang diinginkan.

# Cukup Berkata “Nasi Goreng Satu”

**Analogi**

- **Masalah:** pilihan kasir (Tunai, QRIS, Kartu) sampai berupa teks. Potongan `if` pembuat objeknya cenderung **tersalin** ke banyak tempat.
- Di warung, kamu cukup berkata “nasi goreng satu”. Dapur yang mengurus semuanya, lalu menyerahkan piring yang sudah jadi.
- ***Factory*** memusatkan urusan **membuat objek** di satu tempat. *Factory* yang memutuskan kelas mana yang dibuat.

| Di warung | Pola *Factory* |
|---|---|
| Pembeli menyebut pesanannya | Pemanggil mengirim teks, mis. `"qris"` |
| Dapur memilih bumbu dan cara memasak | *Factory* memilih kelas dan cara membuatnya |
| Piring yang sudah jadi | Objek bertipe `Pembayaran` yang siap dipakai |

# *Simple Factory*: Satu Tempat Memilih Kelas

Pemanggil cukup menulis `KendaraanFactory.buat("motor")` dan menerima objek bertipe `Kendaraan`.

**Python** python/contoh/contoh04_simple_factory.py

```python
class KendaraanFactory:
    @staticmethod
    def buat(jenis: str) -> Kendaraan:
        pilihan = jenis.lower()
        if pilihan == "motor":
            return Motor()
        if pilihan == "mobil":
            return Mobil()
        if pilihan == "mobil besar":
            return MobilBesar()
        # ... (dilewati)
```

**Java** java/contoh/Contoh04SimpleFactory.java

```java
class KendaraanFactory {
    public static Kendaraan buat(String jenis) {
        if (jenis.equalsIgnoreCase("motor")) {
            return new Motor();
        }
        if (jenis.equalsIgnoreCase("mobil")) {
            return new Mobil();
        }
        if (jenis.equalsIgnoreCase("mobil besar")) {
            return new MobilBesar();
        }
        // ... (dilewati)
    }
}
```

# *Simple Factory* dan *Factory Method* Itu Berbeda

| Aspek | *Simple Factory* | *Factory Method* (GoF) |
|---|---|---|
| Bentuk | Satu *method*, biasanya `static`, berisi `if`/`elif` | *Method* di kelas induk yang di-*override* kelas anak |
| Yang memilih kelas produk | *Factory*, berdasarkan parameter | Kelas anak, lewat *overriding* (tanpa `if`) |
| Menambah produk baru | Ubah *factory*: tambah satu cabang `if` | Tambah kelas anak baru; kode lama tidak diubah |
| Termasuk 23 pola GoF? | Tidak, ini idiom yang populer | Ya, kategori *creational* |
| Di materi ini | Contoh 04 dan `PembayaranFactory` | Contoh 05 |

Contoh 05: `EkspedisiDarat` membuat `Truk`, `EkspedisiSungai` membuat `KapalSungai`, lewat `buat_angkutan()` yang di-*override*.

BAGIAN 4

4

# *Strategy*: Algoritma yang Bisa Ditukar

Pola *behavioral* yang sebenarnya sudah kamu pakai sejak Pertemuan 13.

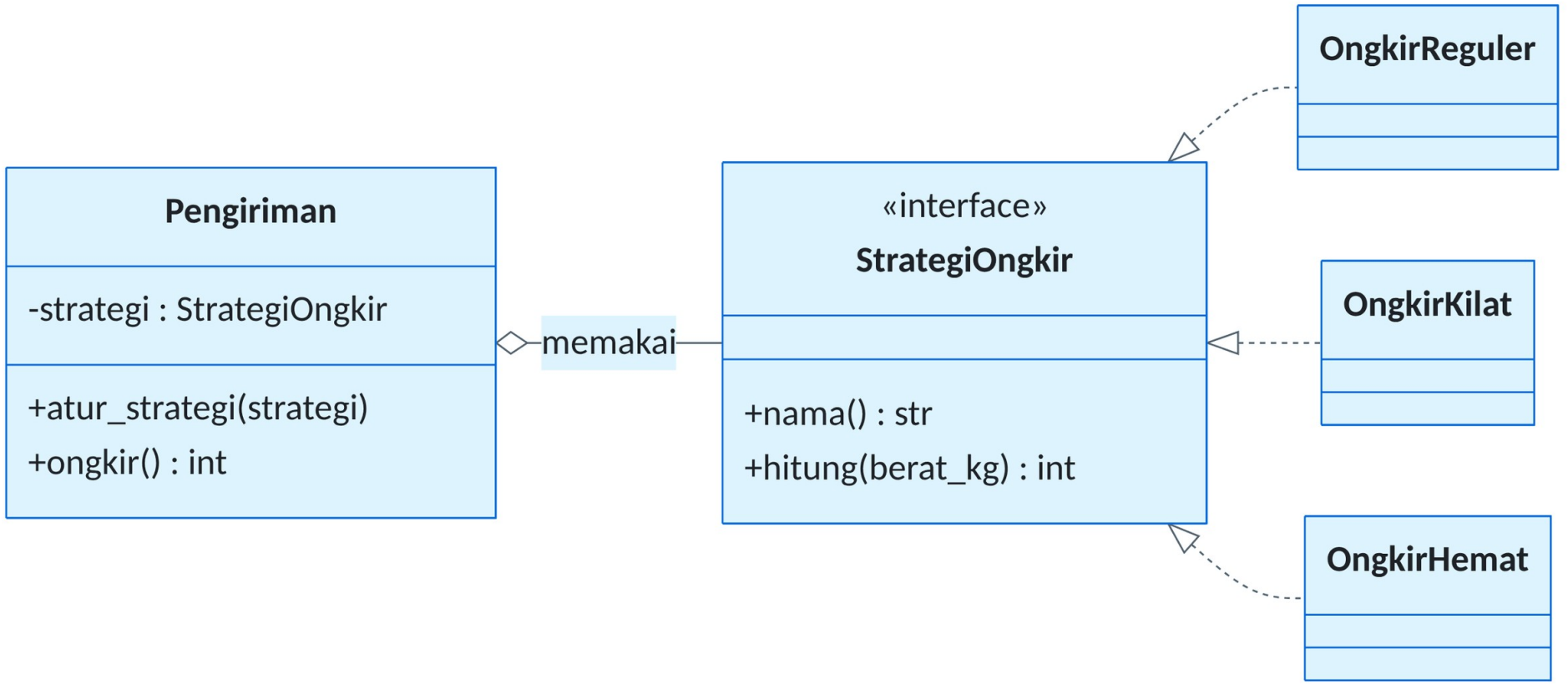
# Rute Tercepat, Terpendek, atau Hindari Tol

### Analogi

- **Masalah:** rumus diskon `KasirSerbaBisa` berupa `if/elif` panjang. Setiap promo baru berarti membongkar *method* yang sama.
- Aplikasi peta bisa menghitung rute **tercepat**, **terpendek**, atau **menghindari tol**, dan pilihannya bisa diganti di tengah perjalanan.
- ***Strategy*** membungkus setiap varian algoritma dalam kelasnya sendiri dengan *interface* yang sama agar bisa **ditukar saat program berjalan**.

| Aplikasi peta | Pola *Strategy* |
|---|---|
| Tujuan perjalanannya sama | Semua strategi memenuhi *interface* yang sama |
| Cara menghitung rutenya berbeda | Setiap rumus punya kelasnya sendiri |
| Diganti di tengah perjalanan | Ditukar saat program berjalan |
| Aplikasi tidak perlu diinstal ulang | Kode yang memakainya tidak diubah |

# Struktur *Strategy*: Contoh Ongkos Kirim




Di KedaiKita: *context* `Pesanan`, kontrak `Diskon`, strategi baru `DiskonHappyHour`, ditukar lewat `atur_diskon()`. Nama bergaya Python.

BAGIAN 5

5

# *Observer*: Kabari Semua yang Berlangganan

Pola *behavioral* kedua: satu kejadian, banyak pihak yang dikabari.

# Tombol Langganan di Kanal Video

**Analogi**

- **Masalah:** setelah Budi membayar, dapur harus mulai memasak dan Budi ingin dikabari lewat SMS. Bulan depan penerimanya bisa bertambah.
- Kamu menekan tombol **langganan** sebuah kanal video. Setiap ada video baru, kanal itu otomatis mengabari semua pelanggannya.
- ***Observer***: *subject* menyimpan daftar pengamat, lalu **otomatis memberi tahu** mereka setiap kali keadaannya berubah.

| Kanal video | Pola *Observer* |
|---|---|
| Kanal yang mengunggah video | ***Subject***: objek yang diamati |
| Kamu, pelanggan kanal itu | Pengamat (***observer***) |
| Notifikasi setiap ada video baru | Kabar otomatis saat keadaan berubah |
| Pemilik kanal tidak tahu siapa pelanggannya | *Subject* tidak tahu kelas konkret pengamat |

# `Pesanan` Mengabari Semua Pengamatnya

Studi kasus KedaiKita

`Pesanan` hanya tahu bahwa setiap pengamat punya `perbarui()`. Ia tidak tahu kelas konkretnya.

**Python** python/studi-kasus/pesanan.py

```python
class Pesanan:
    def __init__(self, pelanggan: Pelanggan):
        # ... (dilewati)
        self._daftar_pengamat: list[PengamatPesanan] = []
    # ... (dilewati)
    def tambah_pengamat(self, pengamat: PengamatPesanan):
        self._daftar_pengamat.append(pengamat)

    def ubah_status(self, status_baru: str):
        # ... (dilewati)
        self._status = status_baru
        for pengamat in self._daftar_pengamat:
            pengamat.perbarui(self)
```

**Java** java/studi-kasus/Pesanan.java

```java
public class Pesanan {
    // ... (dilewati)
    private List<PengamatPesanan> daftarPengamat;
    // ... (dilewati)
    public void tambahPengamat(PengamatPesanan pengamat) {
        daftarPengamat.add(pengamat);
    }
    public void ubahStatus(String statusBaru) {
        // ... (dilewati)
        status = statusBaru;
        for (PengamatPesanan pengamat : daftarPengamat) {
            pengamat.perbarui(this);
        }
    }
    // ... (dilewati)
}
```

# *Push* atau *Pull*?

| Aspek | *Push* (didorong) | *Pull* (ditarik) |
|---|---|---|
| Yang dikirim *subject* | Data kejadiannya, mis. judul video | Objek *subject* itu sendiri |
| Pengamat | Menerima data yang sudah jadi | Mengambil sendiri data yang ia butuhkan |
| Kelebihan | Pengamat tidak perlu mengenal kelas *subject* | Pengamat baru yang butuh data lain tidak mengubah parameter |
| Kekurangan | *Subject* harus menebak data yang dibutuhkan semua pengamat | Pengamat bergantung pada *method-method* milik *subject* |
| Di materi ini | Contoh 07: `perbarui(nama_kanal, judul)` | KedaiKita: `perbarui(pesanan)` |

KedaiKita memakai *pull*: `LayarDapur` menarik nomor dan item, `NotifikasiPelanggan` menarik nama dan nomor HP.

STUDI KASUS



# Studi Kasus KedaiKita

Keempat pola bekerja bersama di kafe kecil di Jl. Pramuka, Samarinda.

# Empat Pola di KedaiKita

Studi kasus KedaiKita

| Pola | Kelas di KedaiKita | Perannya |
|---|---|---|
| *Singleton* | `PengaturanKedai` | Satu sumber nama kedai, alamat, dan pajak 10% |
| *Factory* | `PembayaranFactory` | Membuat objek pembayaran dari teks pilihan kasir |
| *Strategy* | `Diskon` dan kelas-kelas diskonnya, kini ditambah `DiskonHappyHour` | Aturan diskon yang bisa ditukar |
| *Observer* | `PengamatPesanan`, `LayarDapur`, `NotifikasiPelanggan` | Kabar otomatis setiap status pesanan berubah |

Total = subtotal − potongan + pajak, dengan pajak = (subtotal − potongan) × 10%. Persennya dibaca dari `PengaturanKedai`.

# Skenario Budi: Dapur dan Pelanggan Dikabari

Studi kasus KedaiKita

Bagian keempat `main.py`: Budi memesan, membayar tunai lewat *factory*, lalu statusnya diubah tiga kali.

**Python** python/studi-kasus/main.py

```
budi = Pelanggan("Budi Santoso", "081234567890")
pesanan_budi = kedai.buat_pesanan(budi)
pesanan_budi.tambah_item(kedai.cari_menu("M01"), 2)
pesanan_budi.tambah_item(kedai.cari_menu("M03"), 1)
pesanan_budi.tambah_pengamat(LayarDapur())
pesanan_budi.tambah_pengamat(NotifikasiPelanggan())
pesanan_budi.bayar(PembayaranFactory.buat("tunai", 100000, ""))
for status in ["DIPROSES", "SIAP", "SELESAI"]:
    print(f"Status diubah menjadi {status}:")
    pesanan_budi.ubah_status(status)
```

**Khusus Python**

Versi Java ada di `Main.java` dengan *output* yang sama persis.

*Output*

```
...
  [DAPUR] Pesanan baru P002: 2 x Kopi Susu Gula Aren, 1 x Nasi Goreng Kampung
  [SMS ke 081234567890] Halo Budi Santoso, status pesanan P002: LUNAS
...
Status diubah menjadi SELESAI:
  [DAPUR] P002 -> SELESAI
  [SMS ke 081234567890] Halo Budi Santoso, status pesanan P002: SELESAI
```

# Miskonsepsi yang Sering Terjadi

| Anggapan keliru | Yang benar |
| --- | --- |
| “Semakin banyak pola yang dipakai, semakin bagus programnya.” | Pola yang tidak dibutuhkan hanya menambah kelas dan membuat kode sulit diikuti. Pakai pola ketika masalahnya benar-benar ada. |
| “*Singleton* aman dipakai di mana saja karena hanya ada satu objek.” | Justru karena hanya satu, objek itu menjadi *state* global. Perubahan dari satu bagian memengaruhi semua bagian lain, dan pengujian saling mengganggu. |
| “*Strategy* sama saja dengan polimorfisme.” | *Strategy* **memanfaatkan** polimorfisme. Polimorfisme adalah kemampuan bahasa; *Strategy* adalah susunan kelas agar algoritma bisa dipilih dan ditukar oleh *context*. |

# Kuis Cepat

Apa perbedaan *Simple Factory* dan *Factory Method* versi GoF?

**A** *Simple Factory* termasuk 23 pola GoF, sedangkan *Factory Method* tidak

**B** *Factory Method* selalu berupa *method* `static` berisi `if`/`elif`

**C** *Factory Method*: kelas anak meng-*override method* pembuat produk, bukan memilih lewat `if`

**D** Keduanya sama persis, hanya namanya yang berbeda

# Kuis Cepat: Jawaban

Apa perbedaan *Simple Factory* dan *Factory Method* versi GoF?

A. *Simple Factory* termasuk 23 pola GoF, sedangkan *Factory Method* tidak

B. *Factory Method* selalu berupa *method* `static` berisi `if`/`elif`

✓ C. *Factory Method*: kelas anak meng-*override method* pembuat produk, bukan memilih lewat `if`

D. Keduanya sama persis, hanya namanya yang berbeda

**Jawaban: C**

Di contoh 05, `EkspedisiDarat` dan `EkspedisiSungai` masing-masing meng-*override* `buat_angkutan()`/`buatAngkutan()`. A terbalik: *Factory Method* ada di buku GoF, sedangkan *Simple Factory* idiom populer. B justru menggambarkan *Simple Factory*. D salah karena cara memilih produk, cara menambah produk baru, dan statusnya di GoF berbeda.

# Kuis Cepat

Di KedaiKita, `Pesanan` memanggil `pengamat.perbarui(self)` / `pengamat.perbarui(this)`. Gaya *Observer* apa ini, dan mengapa?

**A** *Push* karena pesanan "mendorong" kabar ke semua pengamat

**B** *Push* karena status pesanan dikirim sebagai teks

**C** *Pull* karena *subject* mengirim dirinya sendiri, lalu pengamat mengambil data yang ia butuhkan

**D** *Pull* karena setiap pengamat memeriksa status pesanan berulang-ulang setiap detik

# Kuis Cepat: Jawaban

Di KedaiKita, `Pesanan` memanggil `pengamat.perbarui(self)` / `pengamat.perbarui(this)`. Gaya *Observer* apa ini, dan mengapa?

A. *Push* karena pesanan "mendorong" kabar ke semua pengamat

B. *Push* karena status pesanan dikirim sebagai teks

✓ C. *Pull* karena *subject* mengirim dirinya sendiri, lalu pengamat mengambil data yang ia butuhkan

D. *Pull* karena setiap pengamat memeriksa status pesanan berulang-ulang setiap detik

**Jawaban: C**

`LayarDapur` menarik nomor dan item, `NotifikasiPelanggan` menarik nama dan nomor HP, dari objek pesanan yang sama. Pada *push*, yang dikirim data kejadiannya. A keliru: kedua gaya sama-sama mendorong kabar; bedanya **apa** yang dikirim. B keliru: yang dikirim objek pesanan. D adalah *polling*; pada *Observer*, pengamat **menunggu dikabari**.

# Ringkasan

- ***Design pattern*** adalah solusi teruji untuk masalah desain yang berulang; bukan kode jadi dan jangan dipaksakan.
- Buku **GoF** (1994) memuat **23 pola** dalam 3 kategori: *creational*, *structural*, dan *behavioral*.
- ***Singleton***: satu objek, satu pintu akses `get_instance()`. Kekurangannya: *state* global yang sulit diuji.
- ***Factory*** memusatkan pembuatan objek: *Simple Factory* lewat `if`, *Factory Method* lewat *overriding*.
- ***Strategy*** membuat algoritma bisa ditukar saat program berjalan; hierarki `Diskon` ternyata *Strategy*.
- ***Observer***: *subject* mengabari semua pengamat. *Push* mengirim data, *pull* mengirim *subject*; KedaiKita memakai *pull*.
- KedaiKita kini punya `PengaturanKedai`, `PembayaranFactory`, `DiskonHappyHour`, `LayarDapur`, dan `NotifikasiPelanggan`.

# Langkah Berikutnya

- Jalankan contoh 01—07 dan studi kasus KedaiKita di jalur pilihanmu.
- Ubah pajak menjadi 11% di salinan `main.py` atau `Main.java`; tebak dulu total pesanan Budi dan isi struknya.
- Kerjakan latihan, lalu cek pemahamanmu dengan kuis 10 soal.

**Buka bahan pertemuan ini di GitHub:**

- Konsep
- Praktik Python
- Praktik Java
- Jembatan Python ↔ Java
- Latihan
- Kuis 10 soal

github.com/antonprafanto/PBO2026/tree/master/Pertemuan-14-Design-Pattern

Pindai untuk membuka folder pertemuan ini di GitHub